

Multi-Source Information Driven Spatio-Temporal Hypergraph Learning for Traffic Forecasting

Ping Zhang
School of Artificial Intelligence, Hebei
University of Technology
Tianjin, China
zhangping@hebut.edu.cn

Jiayu Leng
School of Artificial Intelligence, Hebei
University of Technology
Tianjin, China
202432803043@stu.hebut.edu.cn

Liang Yang
School of Artificial Intelligence, Hebei
University of Technology
Tianjin, China
yangliang@vip.qq.com

Anchen Li
Department of Computer Science,
Aalto University
Espoo, Finland
anchen.li@aalto.fi

Xiaochun Cao
School of Cyber Science and
Technology, Shenzhen Campus of Sun
Yat-sen University
Shenzhen, China
caoxiaochun@mail.sysu.edu.cn

Riting Xia*
College of Computer Science, Inner
Mongolia University
Hohhot, China
xiart@imu.edu.cn

Abstract

Accurate traffic flow forecasting is crucial for intelligent transportation systems and relies on effectively modeling complex spatio-temporal dependencies. Although recent graph-based deep learning methods have achieved promising results, most focus on pairwise neighbor relationships, limiting their ability to capture higher-order spatio-temporal interactions in the traffic network. To overcome this limitation, we propose a novel Multi-source information driven Spatio-Temporal HyperGraph learning for traffic forecasting (MSTHG), which is designed to capture richer relational and semantic information. MSTHG introduces a multi-source hypergraph fusion strategy that jointly models dynamic high-order spatial and temporal correlations. Specifically, we build a spatial hypergraph based on geographical proximity to represent high-order spatial dependencies, and a temporal-trend hypergraph leveraging mutual information to capture nonlinear similarities among traffic series. To enhance the semantic richness of node representations, we integrate key daily and weekly information along with periodic features derived from Fast Fourier Transform (FFT). Following the obtained hypergraph, node representations are learned through a hypergraph convolutional network and subsequently processed by a GRU-MLP fusion module, which is designed to capture both local and global temporal dependencies. Extensive experiments on real-world benchmark datasets demonstrate that MSTHG outperforms state-of-the-art baselines. The source code is <https://github.com/April-leng/MSTHG.git>.

CCS Concepts

• Information systems → Spatial-temporal systems; Traffic analysis; • Mathematics of computing → Graph algorithms.

*Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792119>

Keywords

Traffic network; Traffic flow forecasting; Hypergraph learning; Spatio-temporal interactions; Periodicity

ACM Reference Format:

Ping Zhang, Jiayu Leng, Liang Yang, Anchen Li, Xiaochun Cao, and Riting Xia. 2026. Multi-Source Information Driven Spatio-Temporal Hypergraph Learning for Traffic Forecasting. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792119>

1 Introduction

Traffic flow forecasting [18], which serves as a prerequisite and foundation for the Web-driven intelligent transportation system, aims to forecast future traffic conditions utilizing historical traffic flow data and the topology of the road network [20, 25, 31]. Outstanding traffic flow forecasting performance can effectively alleviate various traffic management challenges [9], including road congestion [44], traffic flow control [32] and route planning [33].

The most significant challenge in traffic flow forecasting lies in effectively capturing and modeling the inherent spatio-temporal dependencies within traffic data [25]. Early traffic forecasting methods primarily rely on statistical and traditional machine learning models such as Historical Average (HA) [15], ARIMA [27], and Support Vector Regression (SVR) [11], which provide interpretability but are limited in capturing the nonlinear dynamics of traffic flow. Subsequently, deep learning methods emerge that combine Convolutional Neural Networks (CNNs) with sequential models such as Long Short-Term Memory (LSTM) [16], Gated Recurrent Unit (GRU) [8], and Temporal Convolutional Network (TCN) [46] to jointly capture spatial and temporal traffic patterns. However, these architectures implicitly assume Euclidean grid structures and are unable to model the irregular topology of real-world road networks. To overcome this limitation, advances reformulate traffic systems as graphs and adopt spatio-temporal graph frameworks such as the representative studies DCRNN [26], STGCN [41], and DSTAGNN [24], where graph neural networks (GNNs) are integrated with temporal encoders to more effectively capture dynamic

node interactions over time. Nevertheless, existing spatio-temporal forecasting frameworks still face two fundamental limitations.

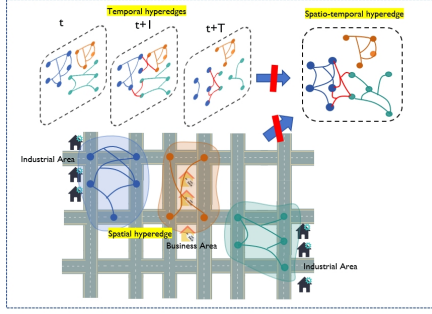


Figure 1: Comparison of different hyperedge construction strategies. As shown, spatial hyperedges are based on proximity, failing to describe the temporal dependencies. Temporal hyperedges connect nodes with similar trends across time slices (e.g., blue and green nodes forming the red hyperedges at $t + T$), but neglect the spatial correlations among neighboring nodes. Spatio-temporal hyperedges jointly encode spatial proximity and temporal similarity.

(1) Unable to capture more complex spatio-temporal dependencies. Hypergraph learning is a powerful tool for modeling high-order, complex spatio-temporal relationships, capable of simultaneously encoding multi-node spatio-temporal interactions through hyperedge construction. The quality of the generated hypergraph structure has a significant impact on traffic forecasting. Some existing methods focus on constructing spatial hypergraphs based on proximity and integrating sequential models to capture temporal dynamics [47]. Other methods emphasize dynamic modeling of temporal hypergraphs by linking nodes with similar traffic patterns within specific time slices to represent evolving high-order relationships [34]. However, the traffic system is simultaneously influenced by the road topology network and dynamic traffic conditions, as shown in Figure 1. Therefore, a more adaptive strategy that jointly integrates spatial and temporal factors to construct spatio-temporal hyperedges is required to better capture complex and dynamic spatio-temporal dependency structures.

(2) Not fully utilize the important temporal information. In traffic forecasting, the temporal behavior of each node is strongly shaped by its own periodic fluctuation patterns. Most existing models either extract periodic signals through implicit temporal embedding modules [5, 47] or design specialized periodic extraction modules, such as the decomposition-based multi-scale strategies [36, 49]. However, such implicit parameter-learning approaches lack the guidance of explicit prior knowledge that incorporates node-specific periodic characteristics, thereby limiting the model's ability to accurately represent the complex temporal dependencies. As a result, we introduce a new periodicity feature to enhance the semantic richness of node representations.

Based on the above analysis, we propose a **Multi-source information driven Spatio-Temporal HyperGraph learning for traffic forecasting (MSTHG)**, which integrates spatial, temporal and periodic information into a unified hypergraph learning architecture.

To tackle the first challenge, we construct a spatio-temporal hypergraph to capture multi-node correlations and model dynamic traffic networks. Specifically, we design a distance-based spatial hypergraph and a mutual information-based temporal hypergraph to construct spatial hyperedges reflecting high-order spatial dependencies and temporal hyperedges capturing nonlinear high-order temporal dependencies, respectively. These two hypergraphs are then fused into a comprehensive hypergraph, where each hyper-edge encodes high-order spatio-temporal interactions among multiple nodes. Adaptive weighting is further applied to enhance the representation capacity of the hypergraph. To address the second challenge, we explicitly quantify node-specific sensitivities to daily and weekly periodic patterns by modeling the inherent periodicity across traffic nodes via Fast Fourier Transform (FFT), thereby improving the robustness of temporal representations under irregular fluctuations. The main contributions are as follows:

- We propose the MSTHG, a novel spatio-temporal hypergraph learning framework for traffic flow forecasting that fuses spatial and temporal hypergraphs to capture dynamic high-order spatio-temporal dependencies.
- We design a periodicity-aware embedding module based on the FFT to extract the periodicity influence degrees of different traffic nodes, serving as explicit priors to enhance the robustness of node temporal representations.
- We evaluate the proposed MSTHG model on four real-world datasets. Through extensive experiments and comparisons with various baseline methods, we demonstrate its superior performance in traffic prediction.

2 Related Work

2.1 Traffic Flow Forecasting

Traffic flow forecasting serves as a fundamental spatio-temporal prediction task in intelligent transportation systems. Early approaches rely on statistical and traditional machine learning models [11, 27], which struggle to model nonlinear dynamics and spatial dependencies. With the rise of deep learning, data-driven neural architectures have become the dominant paradigm due to their superior ability to learn complex spatio-temporal correlations [42]. For instance, FC-LSTM [30] is a classic deep learning architecture that combines LSTM [16] with dense layers to model sequence dependencies. Similarly, TCN [46] employ a stack of dilated causal convolutions to capture long-range temporal dependencies efficiently. Another representative work, LSTNet [23], combines a convolutional layer with a recurrent layer to capture both short-term local dependencies and long-term patterns in multivariate time series. However, such models often assume a Euclidean grid structure, thereby failing to account for the irregular topology of real-world road networks.

To better capture spatial heterogeneity, recent studies use graph neural networks (GNNs) for spatio-temporal modeling [20, 40]. STGCN [41] integrates graph convolution with temporal convolutions. DCRNN [26] models traffic flow as a diffusion process on directed graphs using GRU-based temporal encoding. ASTGCN [14] incorporates attention mechanisms to dynamically weight spatial and temporal dependencies, while AGCRN [3] proposes an adaptive graph generation module to capture fine-grained spatial correlations. To further address the limitations of pre-defined graphs,

GWN [38] develops an adaptive adjacency matrix combined with dilated convolutions to capture latent spatial dependencies. Similarly, MTGNN [37] integrates a graph learning module to extract undirected relations among variables without external graph structures. Focusing on long-term forecasting stability, GMAN [48] adopts an encoder-decoder architecture with spatio-temporal attention mechanisms. Additionally, STNorm [10] introduces a frequency-domain based normalization technique to handle the high-frequency and non-stationary nature of traffic data. Despite their effectiveness, graph-based methods are inherently constrained by pairwise message passing between neighboring nodes, limiting their capacity to capture high-order spatio-temporal interactions [1, 4].

2.2 Hypergraph Construction and Learning

Unlike conventional graph structures, hypergraphs allow hyperedges to connect multiple nodes simultaneously, enabling the modeling of non-pairwise relationships [13]. Recently, hypergraph-based learning has attracted growing interest for modeling complex spatio-temporal dependencies. DHGNN [19] constructs hypergraphs using k-NN and the K-Means algorithm to cluster node features. DHSL [34] dynamically generates hypergraph structures to model high-order correlations among multivariate time series. STHODE [39] constructs two learnable hypergraphs to model the topology of the road network and the dynamics of the traffic separately and integrates the neural ODE modeling to perform traffic forecasting. However, existing methods still struggle to effectively capture interactions between spatial and temporal dimensions when constructing spatio-temporal hypergraphs for prediction.

3 Preliminaries

In this section, we first formalize the problem of traffic flow forecasting. We then introduce the key concepts utilized in our model.

3.1 Problem Definition

For traffic flow forecasting, given the road network $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A})$, where $\mathcal{V}$ is the node set (sensors), $N = |\mathcal{V}|$ is the number of nodes, $\mathcal{E}$ is the edge set representing the connections between nodes, and $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the adjacency matrix with weight describing the spatial proximity (e.g., distance) between nodes.

Our objective is to project future traffic conditions for the next T_{out} time steps from the historical data of the preceding T_{in} time steps by learning a mapping function $f(\cdot)$.

$$[\mathbf{X}_{t-T_{in}+1}, \dots, \mathbf{X}_t, \mathcal{G}] \xrightarrow{f(\cdot)} [\mathbf{X}_{t+1}, \dots, \mathbf{X}_{t+T_{out}}], \quad (1)$$

where $\mathbf{X}_t \in \mathbb{R}^{N \times D}$ denotes the graph signal at time step t . Formally, the historical traffic data is formalized by a graph signal tensor $\mathbf{X} \in \mathbb{R}^{T \times N \times D}$, with T , N , and D denoting the total time steps, number of nodes, and feature dimensions, respectively.

3.2 Hypergraph

A standard graph can only capture pairwise relationships between nodes. To model more complex, higher-order correlations, we introduce the concept of a hypergraph. A hypergraph is a generalization of a graph, denoted as $\mathcal{H} = (\mathcal{V}, \mathcal{E}_h)$, where $\mathcal{V}$ is the set of nodes, and $\mathcal{E}_h$ is the set of hyperedges. Unlike a standard edge, a hyperedge can connect any number of nodes.

The structure of a hypergraph with N nodes and $M = |\mathcal{E}_h|$ hyperedges can be represented by an incidence matrix $\mathbf{H} \in \mathbb{R}^{N \times M}$. The entries of $\mathbf{H}$ are defined as:

$$\mathbf{H}(v, e) = \begin{cases} 1, & \text{if } v \in e \\ 0, & \text{otherwise,} \end{cases} \quad (2)$$

where $v \in \mathcal{V}$ and $e \in \mathcal{E}_h$. The incidence matrix effectively describes the membership of nodes within each hyperedge, forming the foundation for hypergraph-based information propagation.

4 Methodology

This work proposes a novel model named MSTHG for traffic flow forecasting. Our model is designed to comprehensively capture the complex spatio-temporal dependencies inherent in traffic data by integrating multiple specialized components. As shown in Figure 2, the core innovations of our model include: 1) Periodicity-aware embedding module, which enhances the semantic information of node representations with periodicity; 2) Spatio-temporal hypergraph construction module that jointly models high-order spatial and temporal dependencies based on geographical proximity and mutual information; 3) Hypergraph learning & prediction module, which learns the hypergraph representations while simultaneously learning both global and local information in time series, thereby producing more expressive node representations.

4.1 Periodicity-aware Embedding Module

To leverage the valuable information, we introduce the periodicity-aware embedding module to enhance the original data by incorporating both temporal and periodic features, thereby obtaining expressive representations with richer semantics. This module includes feature extraction & embedding, and feature fusion.

4.1.1 Feature Extraction & Embedding. To obtain more comprehensive node representations, this module enriches the raw input by incorporating three distinct feature types. For clarity, we describe these features for all N nodes in a single time step. The three types of features are as follows:

(1) **Raw Traffic Flow:** The original features of the dataset, denoted as $\mathbf{X}^{\text{raw}} \in \mathbb{R}^{N \times D}$.

(2) **Temporal Features:** To provide the time information, we introduce two index matrices: the day-of-week matrix $\mathbf{X}^{\text{dow}} \in \mathbb{Z}^{N \times 1}$ and the time-of-day matrix $\mathbf{X}^{\text{tod}} \in \mathbb{Z}^{N \times 1}$. These two matrices are encoded and concatenated to obtain the matrix $\mathbf{X}^{\text{time}}$.

$$\mathbf{X}^{\text{time}} = \text{Concat}(\mathbf{W}_w \mathbf{X}^{\text{dow}}, \mathbf{W}_d \mathbf{X}^{\text{tod}}), \quad (3)$$

where $\text{Concat}(\cdot)$ is the concatenation function. $\mathbf{W}_w$ and $\mathbf{W}_d$ are weight matrices.

(3) **Periodicity Features:** To explicitly encode the inherent, long-term periodic information, we extract periodicity features from the historical data $\mathbf{X}^{\text{raw}}$. First, we apply the Fast Fourier Transform (FFT) [35] to obtain the frequency spectrum for all nodes:

$$\mathbf{Y}_{\text{FFT}} = \text{FFT}(\mathbf{X}^{\text{raw}}), \quad (4)$$

where $\mathbf{Y}_{\text{FFT}} \in \mathbb{C}^{T \times N} = \{Y_1, Y_2, \dots, Y_T\}$ is the complex-valued frequency spectrum, whose component Y_i is a complex number corresponding to the frequency index i , and its magnitude $|Y_i|$, quantifies the strength of that frequency in the original traffic data.

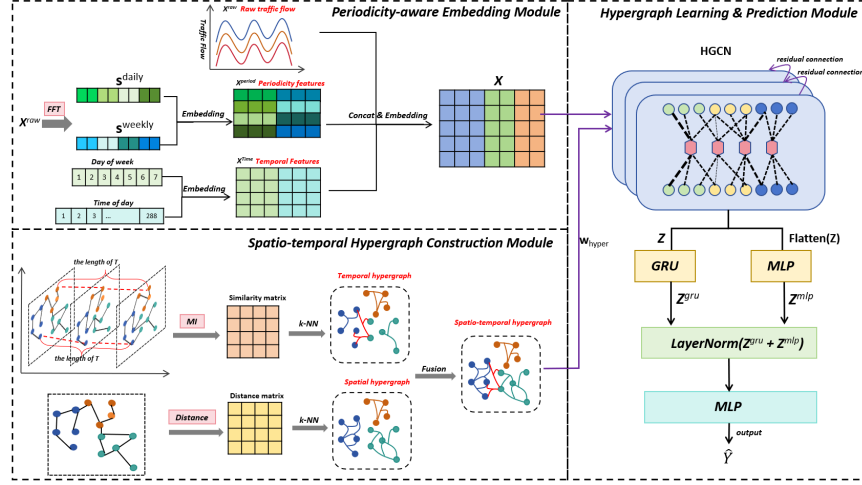


Figure 2: The architecture of MSTHG.

$FFT(\cdot) = \sum_{t=0}^{T-1} x_t \cdot e^{-i \frac{2\pi}{T} kt}$, $k = 0, 1, \dots, T-1$. is the Fast Fourier Transform function. x_t is the traffic flow value of node x at the time step t . The frequency indices k_{daily} and k_{weekly} are determined deterministically based on the sampling rate.

We then quantify the periodic strengths for all nodes by taking the absolute value (magnitude) of the spectrum matrix $\mathbf{Y}_{FFT}$ at these specific frequency indices:

$$\mathbf{s}^{\text{daily}} = |\mathbf{Y}_{k_{\text{daily}}}|, \quad \mathbf{s}^{\text{weekly}} = |\mathbf{Y}_{k_{\text{weekly}}}|. \quad (5)$$

These frequency indices are concatenated and encoded to obtain the final periodicity feature matrix.

$$\mathbf{X}^{\text{period}} = \mathbf{W}_p \text{Concat}(\mathbf{s}^{\text{daily}}, \mathbf{s}^{\text{weekly}}), \quad (6)$$

4.1.2 Feature Fusion. The raw traffic flow features $\mathbf{X}^{\text{raw}}$, temporal features $\mathbf{X}^{\text{time}}$, and periodic features $\mathbf{X}^{\text{period}}$ are integrated to form the final input features for the hypergraph.

$$\mathbf{X}_{\text{fused}} = \text{Concat}(\mathbf{X}^{\text{raw}}, \mathbf{X}^{\text{time}}, \text{Broadcast}(\mathbf{X}^{\text{period}})), \quad (7)$$

where "Broadcast" is a broadcasting operation. We then map the three feature types to a common latent space using an MLP.

$$\mathbf{X} = \text{MLP}(\mathbf{X}_{\text{fused}}), \quad (8)$$

where $\text{MLP}(\cdot)$ is the multilayer perceptron.

In summary, this module transforms the raw traffic flow into a semantically rich, high-dimensional representation by explicitly embedding both its temporal context and periodic information.

4.2 Spatio-temporal Hypergraph Construction Module

To simultaneously capture spatial and temporal dependencies in traffic flow, we introduce a distance-based hypergraph and a mutual information-based temporal hypergraph. The integration of these two structures enables our model to learn more complex dependencies. Furthermore, the module incorporates a weight learning mechanism to dynamically differentiate the importance of hyperedges. It consists of three components: spatial hypergraph learning, temporal hypergraph learning, and weighted hypergraph fusion.

4.2.1 Spatial Hypergraph Learning. The physical distance matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$ is constructed from a pre-defined file containing (u, v, dist) triplets, which represent the distance between sensor u and sensor v . We populate the matrix with these values, setting the diagonal entries to zero and non-adjacent entries to infinity, ensuring the matrix is symmetric. Given the physical distance matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$, we identify a neighborhood set for each node $v_i \in \mathcal{V}$ using the k-Nearest Neighbors (k-NN) algorithm:

$$\mathcal{N}_k(v_i) = \text{k-NN}(\mathbf{D}, v_i, k), \quad (9)$$

where the $\text{k-NN}(\cdot)$ operator returns the set of k nodes closest to v_i . The hyperedge is then formed centered on each node $e_i = \{v_i\} \cup \mathcal{N}_k(v_i)$. After removing duplicate sets, we obtain the spatial hyperedges $\mathcal{E}_{\text{spatial}}$. The spatial hypergraph $\mathbf{H}_{\text{spatial}}$ is

$$\mathbf{H}_{\text{spatial}}(v_i, e_j) = \begin{cases} 1, & \text{if } v_i \in e_j \\ 0, & \text{otherwise,} \end{cases} \quad (10)$$

where $e_j \in \mathcal{E}_{\text{spatial}}$ is the hyperedge.

4.2.2 Temporal Hypergraph Learning. We first compute a pairwise similarity matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ by estimating the Mutual Information (MI) [22] between the time series of every node pair:

$$S_{ij} = \text{MI}(\mathbf{X}_{:,i}, \mathbf{X}_{:,j}), \quad (11)$$

$$\text{MI}(\mathbf{X}_{:,i}; \mathbf{X}_{:,j}) = \sum_{k_1=1}^T \sum_{k_2=1}^T p(x_{k_1,i}, x_{k_2,j}) \log \left(\frac{p(x_{k_1,i}, x_{k_2,j})}{p(x_{k_1,i})p(x_{k_2,j})} \right), \quad (12)$$

where $p(x_{k_1,i}, x_{k_2,j})$ is the joint probability function of observing values from the time series of node i and node j , while $p(x_{k_1,i})$ and $p(x_{k_2,j})$ are the respective marginal probability functions. Similar to the spatial construction, a temporal hyperedge e'_i is formed for each node v_i by grouping it with its k most functionally similar neighbors. After removing duplicates, we get the unique set of temporal hyperedges. The temporal hypergraph $\mathbf{H}_{\text{temporal}}$ is

$$\mathbf{H}_{\text{temporal}}(v_i, e'_j) = \begin{cases} 1, & \text{if } v_i \in e'_j \\ 0, & \text{otherwise,} \end{cases} \quad (13)$$

where $e'_j \in \mathcal{E}_{\text{temporal}}$ is the hyperedge.

4.2.3 Structure Fusion and Adaptive Weighting. The final hypergraph structure $\mathbf{H}$ is formed by the union of the hypergraph structure from both $\mathbf{H}_{\text{spatial}}$ and $\mathbf{H}_{\text{temporal}}$. Specifically, the hyperedges $\mathcal{E}_h$ is the union of $\mathcal{E}_{\text{spatial}}$ and $\mathcal{E}_{\text{temporal}}$. To allow the model to learn the importance of these relationships, we initialize a learnable weight vector $\mathbf{w}_{\text{hyper}} \in \mathbb{R}^M$, which acts as an adaptive mask on the hyperedges during the learning process. M is the total number of unique hyperedges in the fused hypergraph. The hypergraph $\mathbf{H}$ is

$$\mathbf{H}(v_i, e''_j) = \begin{cases} w_i, & \text{if } v_i \in e''_j \\ 0, & \text{otherwise,} \end{cases} \quad (14)$$

where e''_j is the hyperedge. Overall, by integrating temporal hypergraphs and spatial hypergraphs, we obtain the final hypergraph structure $\mathbf{H}$, capturing more complex dependencies.

4.3 Hypergraph Learning & Prediction Module

To obtain node representations of the hypergraph that capture both global and local information, we introduce a hypergraph learning module with GRU and MLP to learn local and global information. These refined representations are utilized by the prediction module to learn and infer future traffic flow.

4.3.1 Hypergraph Learning. We first introduce the hypergraph convolution network (HGCN) to learn node representations.

$$\mathbf{Z} = \text{HGCN}(\mathbf{X}, \mathbf{H}). \quad (15)$$

The HGCN layer performs spatio-temporal information propagation based on the hypergraph convolution, as introduced in Hypergraph Neural Networks [12]. The process of layer l is formulated as a two-stage message passing mechanism.

First, node representations $\mathbf{X}^{(l-1)}$, are aggregated to form intermediate hyperedge representations $\mathbf{X}^{(l)}$:

$$\mathbf{X}^{(l)} = \mathbf{D}_e^{-1} \mathbf{H}^T \mathbf{X}^{(l-1)}, \quad (16)$$

where $\mathbf{X}^{(l-1)} \in \mathbb{R}^{N \times d}$ is the input node feature matrix from the previous layer, $\mathbf{H} \in \{0, 1\}^{N \times M}$ is the hypergraph structure matrix, and $\mathbf{D}_e \in \mathbb{R}^{M \times M}$ is the diagonal matrix of hyperedge degrees. These hyperedge representations are then weighted and propagated back to update the node representations for the current layer $\mathbf{X}^{(l)}$:

$$\mathbf{X}^{(l)} = \mathbf{H}(\mathbf{w}_{\text{hyper}} \odot \mathbf{X}^{(l)}), \quad (17)$$

where $\mathbf{w}_{\text{hyper}} \in \mathbb{R}^M$ is the learnable hyperedge weight vector, and $\odot$ denotes the element-wise broadcast multiplication.

To mitigate potential vanishing gradient issues and preserve original feature information, we introduce residual connections. This is achieved by feeding the concatenation of the original features $\mathbf{X}$ and the aggregated features $\mathbf{X}^{(l)}$ into an MLP, and then adding this transformed output back to the original features. The expression is as follows:

$$\mathbf{X}^{(l+1)} = \text{LayerNorm}(\mathbf{X} + \text{MLP}(\text{Concat}(\mathbf{X}, \mathbf{X}^{(l)}))), \quad (18)$$

where $\text{LayerNorm}(\cdot)$ is the layer normalization for stability.

We then introduce the parallel hybrid mechanism to decode the sequence of node representations $\mathbf{Z}$, which includes the GRU and

MLP decoder. Specifically, for the GRU branch, we use GRU to learn the local dependencies within the time series.

$$\mathbf{Z}^{\text{gru}} = \text{GRU}(\mathbf{Z}), \quad (19)$$

where the internal update mechanism of the GRU for an input $\mathbf{z}_t$ at each time step t is defined by the following equations:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{z}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r), \quad (20)$$

$$\mathbf{g}_t = \sigma(\mathbf{W}_g \mathbf{z}_t + \mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{b}_g), \quad (21)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{z}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h), \quad (22)$$

$$\mathbf{h}_t = \mathbf{g}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{g}_t) \odot \tilde{\mathbf{h}}_t, \quad (23)$$

where $\mathbf{h}_{t-1}$ is the hidden state from the previous time step. $\mathbf{r}_t$, $\mathbf{g}_t$, and $\tilde{\mathbf{h}}_t$ are the reset gate, update gate, and candidate hidden state, respectively. σ denotes the sigmoid function, and $\odot$ represents the element-wise Hadamard product. The $\mathbf{W}_r$, $\mathbf{W}_g$, $\mathbf{W}_h$, $\mathbf{U}_r$, $\mathbf{U}_g$, $\mathbf{U}_h$, and $\mathbf{b}_r$, $\mathbf{b}_g$, $\mathbf{b}_h$ are learnable parameters. The final output $\mathbf{Z}^{\text{gru}}$ is the last hidden state $\mathbf{h}_{T_{in}}$ after the GRU processes the entire sequence of length T_{in} .

For temporal MLP branch, we use MLP to capture the global information of the time series.

$$\mathbf{Z}^{\text{mlp}} = \text{MLP}_{\text{temp}}(\text{Flatten}(\mathbf{Z})), \quad (24)$$

where $\text{Flatten}(\cdot)$ reshapes the tensor to a single vector, and MLP_{temp} is the temporal MLP.

These two representations are fused via element-wise addition to produce the final node representation $\mathbf{Z}^{\text{final}}$.

$$\mathbf{Z}^{\text{final}} = \text{LayerNorm}(\mathbf{Z}^{\text{gru}} + \mathbf{Z}^{\text{mlp}}). \quad (25)$$

4.3.2 Traffic Flow Prediction Module. We use two-layer MLP to predict the traffic flow for the next T_{out} time steps.

$$\hat{\mathbf{Y}} = \text{MLP}_{\text{out}}(\mathbf{Z}^{\text{final}}), \quad (26)$$

where MLP_{out} is the two-layer MLP, and $\hat{\mathbf{Y}}$ is the final prediction tensor for the next T_{out} time steps.

4.4 Model Optimization

To train our model, we design a joint loss function that comprises two distinct components: a traffic flow prediction loss and a structural consistency loss. The general objective is to minimize prediction loss while constraining the learned hypergraph structure.

To regularize the weights and prevent the learned structure from deviating too drastically from the prior, we introduce the MSE loss $\mathcal{L}_{\text{consist}}$ between the learnable weights and their static prior.

$$\mathcal{L}_{\text{consist}} = \frac{1}{M} \sum_{i=1}^M (\mathbf{w}_{\text{hyper},i} - \mathbf{w}_{\text{prior},i})^2, \quad (27)$$

where $\mathbf{w}_{\text{hyper}}$ is the learnable hyperedge weight vector. $\mathbf{w}_{\text{prior}}$ is the static prior weight vector for initialization, where each element is 1. This loss acts as a regularization term, encouraging the proposed model to make necessary adjustments to the initial hypergraph structure.

The objective of our model is to accurately forecast future traffic flow. To this end, we employ the Masked Mean Absolute Error

Table 1: Forecasting errors on PEMS03, PEMS04, PEMS07 and PEMS08 datasets.

Model	PeMS03			PeMS04			PeMS07			PeMS08		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	31.58	52.39	33.78	38.03	59.24	27.88	45.12	65.64	24.51	34.86	59.24	27.88
ARIMA	26.33	33.05	22.90	28.55	40.36	19.55	33.89	46.38	17.60	31.23	33.47	19.25
FC-LSTM	21.33	35.11	23.33	26.77	40.65	18.23	29.98	45.94	13.20	23.09	35.17	14.99
GRU-ED	19.12	32.85	19.31	23.68	39.27	16.44	27.66	43.49	12.20	22.00	36.22	13.33
TCN	19.32	33.55	19.93	23.22	37.26	15.59	32.72	42.23	14.26	22.72	35.79	14.03
DSANet	21.29	34.55	23.21	22.79	35.77	16.03	31.36	49.11	14.43	17.14	26.96	11.32
STGCN	17.55	30.42	17.34	21.16	34.89	13.83	25.33	39.34	11.21	17.50	27.09	11.29
DCRNN	18.18	30.31	18.34	24.70	38.12	17.12	25.30	38.58	11.66	17.86	27.83	11.45
GWN	19.12	32.77	18.89	24.89	39.66	17.29	26.39	41.50	11.97	18.28	30.05	12.15
ASTGCN	17.34	29.56	17.21	22.93	35.22	16.56	24.01	37.87	10.73	18.25	28.06	11.64
STG2Seq	19.03	29.83	21.55	25.20	38.48	18.77	32.77	47.16	20.16	20.17	30.71	17.32
Z-GCNETs	16.64	28.15	16.39	19.50	31.61	12.78	21.77	35.17	9.25	15.76	25.11	10.01
STID	15.64	26.38	15.46	19.58	31.79	13.38	21.52	36.29	9.15	15.58	25.89	10.33
STGHCN	14.85	25.45	15.85	19.40	31.04	13.42	21.18	34.20	9.68	15.36	24.52	10.28
GLMST	15.21	26.77	14.74	19.00	31.20	12.46	20.73	34.60	8.76	15.48	24.69	9.78
LightST	15.11	24.74	14.41	19.21	31.31	12.70	20.78	33.95	8.98	15.43	24.52	9.84
MSTHG	14.73	23.70	14.36	18.43	28.33	12.40	19.76	32.15	8.24	13.67	22.44	9.17

(MAE) as the prediction loss. This loss function calculates the absolute difference between the model's predictions $\hat{Y}$ and the ground truth values Y . The formulation is as follows:

$$\mathcal{L}_{pred} = \|\hat{Y} - Y\|, \quad (28)$$

where $\hat{Y}$ and Y are the predicted and ground truth values. The overall loss function is:

$$\mathcal{L}_{total} = \mathcal{L}_{pred} + \mu \cdot \mathcal{L}_{consist}, \quad (29)$$

where μ is a hyperparameter that balances the trade-off between prediction accuracy and structural regularization. By minimizing this joint loss, we train the model to make accurate predictions while maintaining a stable and meaningful hypergraph structure.

5 Experiments

In this section, we first introduce the experimental settings, covering the datasets, baselines, and implementation details. We then present the experimental results, which include performance comparisons on traffic flow forecasting, ablation study, case study, time efficiency analysis, hyper-parameter sensitivity analysis, and analysis of dynamic hypergraph structure learning. These components facilitate a deeper understanding of the proposed model.

5.1 Experimental Settings

5.1.1 Datasets. We evaluate our MSTHG on several widely used traffic datasets (PEMS03, PEMS04, PEMS07, and PEMS08)¹. The raw traffic data is aggregated into 5-minute time steps. Dataset details are shown in Table 2.

5.1.2 Baselines. We select three categories of traffic flow forecasting prediction baselines, particularly those that have been recognized as the state-of-the-art. (1) For classic statistical methods, we

¹<https://pems.dot.ca.gov/>.

Table 2: Statistics of traffic datasets.

Dataset	Time Range	Time Steps	E	V
PEMS08	07/2016 - 08/2016	17,856	295	170
PEMS07	05/2017 - 08/2017	28,224	866	883
PEMS04	01/2018 - 02/2018	16,992	340	307
PEMS03	09/2018 - 11/2018	26,208	547	358

compare (a) HA [15], which predicts using historical data averages; (b) ARIMA [27], which is a classic statistical time-series model. (2) For deep learning methods without graphs, we compare (a) FC-LSTM [30], which uses LSTM [16] with fully connected units; (b) TCN [46], which employs stacked dilated causal convolutions; (c) GRU-ED [7], which uses a GRU-based encoder-decoder architecture; (d) DSANet [17], which combines CNN with dual self-attention; (e) STID [29], which is an MLP-based model with spatio-temporal embeddings. (3) For deep learning methods with the graphs, we compare (a) STGCN [41], which combines graph convolution and temporal convolution; (b) DCRNN [26], which integrates diffusion convolution into a GRU; (c) GWN [38], which uses diffusion GCN with adaptive graphs; (d) ASTGCN [14], which employs spatio-temporal attention mechanisms; (e) STG2Seq [2], which is a seq2seq model with gated GCN; (f) Z-GCNETs [6], which tracks topological features using zigzag persistence; (g) STGHCN [5], which utilizes both gated graph convolution and hypergraph convolution to capture spatial patterns; (h) GLMST [45], which proposes a pure MLP-based architecture without relying on graph convolution to capture spatial dependencies; (i) LightST [43], which is a lightweight spatio-temporal mixing model.

5.1.3 Implementation Details. The model is implemented based on the PyTorch [28], and all experiments are trained and evaluated on a single NVIDIA GeForce RTX 4090 GPU. Our model leverages

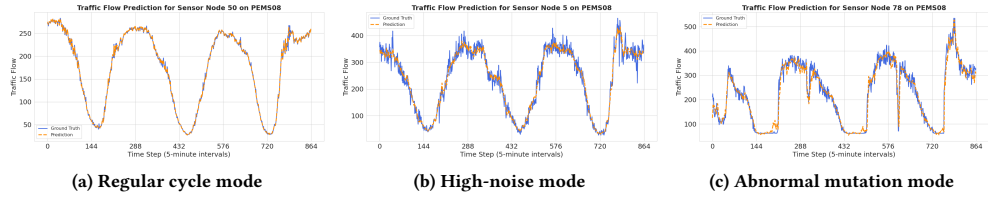


Figure 3: The predictive ability of our model in different traffic scenarios.

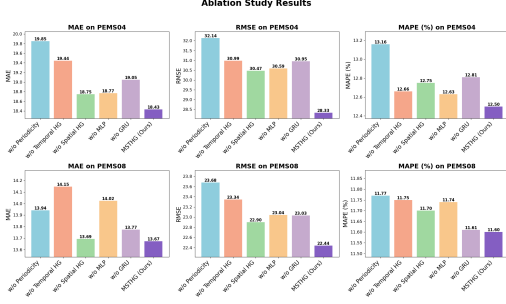


Figure 4: Ablation study on PEMS04 and PEMS08.

a sequence of 12 historical time steps to forecast the subsequent 12 time steps. The datasets are chronologically partitioned into training sets (60%), validation sets (20%), and test sets (20%). We apply the Z-Score method to standardize the input data. For **hypergraph construction module**, the number of neighbors k is set to 8 for both spatial and temporal hypergraphs. The construction of a spatio-temporal hypergraph is a one-time offline preprocessing step. For example, it takes approximately 126.32 seconds to construct a spatio-temporal hypergraph on the PEMS08 dataset. The computational investment is reasonable and it provides rich prior knowledge. We use the flow as characteristic value, and $D = 1$. For **periodicity-aware embedding module**, the dimensions of the temporal feature embedding and the periodicity feature embedding are 16. For **hypergraph learning & prediction module**, the MLP layer is 2 and the hidden dimension is 256. The final dimension of node representation is 12. In terms of optimization, we use the Adam optimizer [21] to train the proposed model. The batch size is 32. The learning rate is set to 0.0005, and the total number of epochs is 110.

5.2 Experimental Results

5.2.1 The Performance of MSTHG. We conduct a comprehensive performance comparison of the proposed MSTHG with baselines on four public datasets. The details are shown in Table 1.

Based on these results, we have drawn the following observations. (1) **The performance of traditional statistical methods is far inferior to that of deep learning methods.** This is because these methods are typically based on linear assumptions, rendering them incapable of capturing the inherent high non-linearity in traffic flow dynamics. Concurrently, their reliance on stationarity assumptions is often violated in real-world traffic scenarios, such as

during peak hours and holidays. (2) **Graph-based deep learning methods outperform traditional deep learning approaches.** While traditional deep learning models outperform traditional methods due to their nonlinearity, they are spatially agnostic and cannot utilize the physical connectivity between sensors. In contrast, graph-based deep learning methods explicitly encode the road network topology into a graph structure. By utilizing message-passing mechanisms, they can simulate the spatial propagation of traffic conditions (e.g., congestion), thereby enabling the synchronous modeling of spatio-temporal dependencies. (3) **Our model outperforms all baselines across nearly all benchmarks.** MSTHG achieves the best performance in MAE and RMSE on all datasets, and also demonstrates extremely strong competitiveness in the MAPE metric. The performance advantage is attributed to three aspects of the model: i) By integrating spatial hypergraphs based on physical distance with temporal hypergraphs based on MI, the model can capture more complex temporal and spatial dependencies. ii) Introduce periodic features based on FFT to enable the model to explicitly learn the inherent daily and weekly patterns in traffic flow, which is a key piece of information overlooked by many models. iii) An efficient modular architecture can hierarchically extract and fuse spatio-temporal features, thereby generating more robust node representations.

5.2.2 Ablation Study. To verify the effectiveness of each component in the MSTHG, we conduct an ablation study by removing individual modules and comparing our model MSTHG with its variants: w/o Periodicity (the periodic embedding module based on FFT extraction is removed), w/o Temporal HG (removing the temporal hypergraph), w/o Spatial HG (removing the spatial hypergraph), w/o GRU or w/o MLP (no GRU or MLP module). The experiments are conducted on the PEMS04 and PEMS08 datasets, and the results are shown in Figure 4. The results demonstrate that removing any key component from MSTHG compromises its accuracy, confirming that each component, including periodic feature extraction, dual hypergraph fusion, and the dual-branch time-series architecture, is integral to achieving its state-of-the-art performance.

5.2.3 Case Study. To more intuitively evaluate the predictive ability of our model in different traffic scenarios, this section presents the visualization of the prediction results of several representative sensor nodes on the PEMS08 dataset and compares them with the true values, as shown in the Figure 3. We deliberately select three different types of traffic modes: the regular cycle mode, the high-noise mode, and the abnormal mutation mode.

Based on the results, we have drawn the following observations. (a) **For regular cycle mode (as shown in Figure 3(a))**, the traffic

flow prediction of sensor 50 over several consecutive days. The traffic pattern of this sensor exhibits relatively regular periodicity, with clear morning and evening rush hours and night troughs. The curve is smooth and easy to predict. As can be seen from the figure, the prediction curve of our model (the orange dotted line) highly coincides with the true value curve (the blue solid line). Whether it is the size and occurrence time of the peak or the depth of the trough, all have been precisely captured. This indicates that our model can learn and fit conventional and predictable traffic patterns. (b) **For high-noise mode (as shown in Figure 3(b))**, corresponding to the prediction results of sensor 5, this is a more challenging scenario. Although the traffic flow of this sensor also has daily periodicity, its real signal contains a large amount of random noise and high-frequency fluctuations, resulting in many "burrs" on the curve. Our model accurately captures the overall trend of traffic flow, including the rise and fall of peak hours. This demonstrates that our model is robust and can effectively extract latent patterns from noisy signals. (c) **For abnormal mutation mode(as shown in Figure 3(c))**, we present a strange and complex pattern of sensor 78. During peak hours, the traffic flow will drop sharply to a very low level in a "cliff-like" manner and remain in an abnormal "stable period" rather than a smooth transition. This mutation pattern may be caused by special events such as traffic accidents, road control or sensor malfunctions. It is worth noting that our model successfully predicted the sharp decline and prolonged low-traffic state, quickly responding to the next peak. It accurately identified sudden changes despite minor stable-period fluctuations, proving its adaptability and generalization ability for abnormal traffic patterns.

Table 3: Runtime comparison on different datasets.

Dataset	PEMS04	PEMS07	PEMS08
Methods	Seconds/epoch		
DCRNN	95.12	510.53	57.17
STGCN	41.16	198.13	25.31
GWN	27.88	1710.6	19.72
AGCRN	28.49	189.5	19.29
StemGNN	16.29	136.31	9.41
GMAN	107.31	827.77	71.04
MTGNN	25.11	107.03	79.45
STNorm	18.20	74.12	32.45
STID	5.24	14.32	4.46
MSTHG(ours)	26.47	129.83	18.55

5.2.4 Time Efficiency Analysis. we compare the average training time for each epoch of our model and baselines. Specifically, we evaluate the overall time consumption of both the MSTHG and baselines on the PEMS08, PEMS07 and PEMS04. The results are shown in Table 3. As shown in Table 3, the running time of MSTHG is acceptable.

5.2.5 Analysis of the Dynamic Hypergraph Structure Learning. We analyze the hyperedge weights to examine the physical implications of the higher-order structures. The hyperedge weight reflects the model's assessment of a node combination's importance to the

overall prediction task. We select the three highest-weight hyperedges and the two lowest-weight hyperedges, then visualize the nodes they contain in the real road network topology. The results are shown in Figure 5.

We observe that (1) **High-weight hyperedges reveal "functional traffic paths"**. Figure 5 (left) illustrates that high-weighted nodes (e.g., orange and green clusters) confirms the model can capture long-range dependencies and traffic pattern similarity between geographically distant nodes, surpassing the limitations of mere physical proximity. (2) **Low-weight hyperedge reflects "the redundancy of local information"**. Figure 5 (right) illustrates that lowest-weighted nodes(e.g., Blue and pink clusters) exhibit highly localized clustering, confirming the model efficiently suppresses redundancy information.

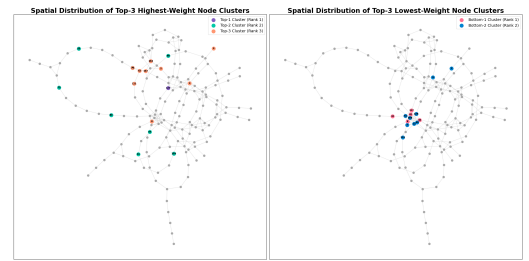


Figure 5: Visualize the learned hyperedge weights on the PEMS08.

6 Conclusion

In this paper, we propose a novel method, MSTHG, a multi-source information driven spatio-temporal hypergraph learning for traffic flow forecasting. Specifically, we introduce the key daily and weekly information along with periodic features derived from Fast Fourier Transform (FFT) to obtain semantic richness of node representations. Furthermore, we propose a spatio-temporal hypergraph construction module to obtain spatial and temporal hypergraphs. Following the obtained hypergraph, node representations are learned through a hypergraph convolutional network and subsequently processed by a GRU-MLP fusion module, which is designed to capture both local and global temporal dependencies. Experimental results on traffic flow datasets demonstrate the superior performance of MSTHG compared to baselines for traffic forecasting.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Nos. 92570118, 62506177, 62376088, and 62206085); The Inner Mongolia Autonomous Region Natural Science Foundation (No. 2025QN06010); Hebei Natural Science Foundation (Nos. F2025202050, and F2024202047); Hebei Yanzhao Golden Platform Talent Gathering Programme Core Talent Project (Education Platform) (HJZD202509); Hebei Yanzhao Golden Platform Talent Gathering Program Core Talent Project (Postdoctoral Platform) (No. B2024005001).

References

- [1] Alessia Antelmi, Gennaro Cordasco, Mirko Polato, Vittorio Scarano, Carmine Spagnuolo, and Dingqi Yang. 2024. A Survey on Hypergraph Representation Learning. *ACM Comput. Surv.* 56, 1 (2024), 24:1–24:38.
- [2] Lei Bai, Lina Yao, Salil S. Kanhere, Xianzhi Wang, and Quan Z. Sheng. 2019. STG2Seq: Spatial-Temporal Graph to Sequence Model for Multi-step Passenger Demand Forecasting. In *IJCAI*. 1981–1987.
- [3] Lei Bai, Lina Yao, Can Li, Xianzhi Wang, and Can Wang. 2020. Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting. In *NeurIPS*.
- [4] Austin R. Benson, David F. Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.
- [5] Shuqin Cao, Libing Wu, Rui Zhang, Yanjiao Chen, Jianxin Li, and Qin Liu. 2024. A Spatial-Temporal Gated Hypergraph Convolution Network for Traffic Prediction. *IEEE Trans. Veh. Technol.* 73, 7 (2024), 9546–9559.
- [6] Yuzhou Chen, Ignacio Segovia-Dominguez, and Yulia R. Gel. 2021. Z-GCNets: Time Zigzags at Graph Convolutional Networks for Time Series Forecasting. In *ICML*, Vol. 139. 1684–1694.
- [7] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the Properties of Neural Machine Translation: Encoder-Decoder Approaches. In *SSST@EMNLP*. 103–111.
- [8] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *EMNLP*. 1724–1734.
- [9] Rui Dai, Shenkun Xu, Qian Gu, Chenguang Ji, and Kaikui Liu. 2020. Hybrid Spatio-Temporal Graph Convolutional Network: Improving Traffic Prediction with Navigation Data. In *KDD*. 3074–3082.
- [10] Jinliang Deng, Xiusi Chen, Renhe Jiang, Xuan Song, and Ivor W. Tsang. 2021. ST-Norm: Spatial and Temporal Normalization for Multi-variate Time Series Forecasting. In *KDD*. 269–278.
- [11] Harris Drucker, Christopher J. C. Burges, Linda Kaufman, Alexander J. Smola, and Vladimir Vapnik. 1996. Support Vector Regression Machines. In *NeurIPS*. 155–161.
- [12] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph Neural Networks. In *AAAI*. 3558–3565.
- [13] Yue Gao, Zizhao Zhang, Haojie Lin, Xibin Zhao, Shaoyi Du, and Changqing Zou. 2022. Hypergraph Learning: Methods and Practices. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 5 (2022), 2548–2566.
- [14] Shengnan Guo, Youfang Lin, Ning Feng, Chao Song, and Huaiyu Wan. 2019. Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting. In *AAAI*. 922–929.
- [15] Paul Harrison. 1999. Statistical Visions in Time: A History of Time Series Analysis 1662-1938 By Judy L. Klein. Cambridge, U.K.: Cambridge University Press, 1997. Pp. xix, 345. \$64.95. *Southern Economic Journal* 66, 1 (1999), 200–202.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780.
- [17] Siteng Huang, Donglin Wang, Xuehan Wu, and Ao Tang. 2019. DSANet: Dual Self-Attention Network for Multivariate Time Series Forecasting. In *CIKM*. 2129–2132.
- [18] Jiahao Ji, Jingyuan Wang, Zhe Jiang, Jiawei Jiang, and Hu Zhang. 2022. STDEN: Towards Physics-Guided Neural Networks for Traffic Flow Prediction. In *AAAI*. 4048–4056.
- [19] Jianwen Jiang, Yuxuan Wei, Yifan Feng, Jingxuan Cao, and Yue Gao. 2019. Dynamic Hypergraph Neural Networks. In *IJCAI*. 2635–2641.
- [20] Weiwei Jiang and Jiayun Luo. 2022. Graph neural network for traffic forecasting: A survey. *Expert Syst. Appl.* 207 (2022), 117921.
- [21] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *ICLR*.
- [22] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. 2004. Estimating mutual information. *Physical Review E* 69 (2004), 066138. Issue 6.
- [23] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. 2018. Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. In *SIGIR*. 95–104.
- [24] Shiyong Lan, Yitong Ma, Weikang Huang, Wenwu Wang, Hongyu Yang, and Pyang Li. 2022. DSTAGNN: Dynamic Spatial-Temporal Aware Graph Neural Network for Traffic Flow Forecasting. In *ICML*, Vol. 162. 11906–11917.
- [25] Fuxian Li, Jie Feng, Huan Yan, Guangyin Jin, Fan Yang, Funing Sun, Depeng Jin, and Yong Li. 2023. Dynamic Graph Convolutional Recurrent Network for Traffic Prediction: Benchmark and Solution. *ACM Trans. Knowl. Discov. Data* 17, 1 (2023), 9:1–9:21.
- [26] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *ICLR*.
- [27] Brian K. Nelson. 1998. Time Series Analysis Using Autoregressive Integrated Moving Average (ARIMA) Models. *Academic Emergency Medicine* 5, 7 (1998), 739–744.
- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*. 8024–8035.
- [29] Zezhi Shao, Zhao Zhang, Fei Wang, Wei Wei, and Yongjun Xu. 2022. Spatial-Temporal Identity: A Simple yet Effective Baseline for Multivariate Time Series Forecasting. In *CIKM*. 4454–4458.
- [30] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to Sequence Learning with Neural Networks. In *NeurIPS*. 3104–3112.
- [31] David Alexander Tedjopurnomo, Zhifeng Bao, Baihua Zheng, Farhana Murtaza Choudhury, and Alex Kai Qin. 2022. A Survey on Modern Deep Neural Network for Traffic Prediction: Trends, Methods and Challenges. *IEEE Trans. Knowl. Data Eng.* 34, 4 (2022), 1544–1561.
- [32] Jingyuan Wang, Jiawei Jiang, Wenjun Jiang, Chao Li, and Wayne Xin Zhao. 2021. LibCity: An Open Library for Traffic Prediction. In *SIGSPATIAL*. 145–148.
- [33] Senzhang Wang, Jiannong Cao, and Philip S. Yu. 2022. Deep Learning for Spatio-Temporal Data Mining: A Survey. *IEEE Trans. Knowl. Data Eng.* 34, 8 (2022), 3681–3700.
- [34] Shun Wang, Yong Zhang, Xuanqi Lin, Yongli Hu, Qingming Huang, and Baocai Yin. 2024. Dynamic Hypergraph Structure Learning for Multivariate Time Series Forecasting. *IEEE Trans. Big Data* 10, 4 (2024), 556–567.
- [35] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *ICLR*.
- [36] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *NeurIPS*. 22419–22430.
- [37] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaoju Chang, and Chengqi Zhang. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *KDD*. 753–763.
- [38] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. In *IJCAI*. 1907–1913.
- [39] Chengzhi Yao, Zhi Li, and Junbo Wang. 2023. Spatio-Temporal Hypergraph Neural ODE Network for Traffic Forecasting. In *ICDM*. 1499–1504.
- [40] Jie Xia Ye, Juanjuan Zhao, Kejiang Ye, and Cheng-Zhong Xu. 2022. How to Build a Graph-Based Deep Learning Architecture in Traffic Domain: A Survey. *IEEE Trans. Intell. Transp. Syst.* 23, 5 (2022), 3904–3924.
- [41] Bing Yu, Haoqiang Yin, and Zhanxing Zhu. 2018. Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *IJCAI*. 3634–3640.
- [42] Junbo Zhang, Yu Zheng, and Dekang Qi. 2017. Deep Spatio-Temporal Residual Networks for Citywide Crowd Flows Prediction. In *AAAI*. 1655–1661.
- [43] Qianru Zhang, Xinyi Gao, Haixin Wang, Siu Ming Yiu, and Hongzhi Yin. 2025. Efficient Traffic Prediction Through Spatio-Temporal Distillation. In *AAAI*. 1093–1101.
- [44] Xiyue Zhang, Chao Huang, Yong Xu, and Lianghao Xia. 2020. Spatial-Temporal Convolutional Graph Attention Networks for Citywide Traffic Flow Forecasting. In *CIKM*. 1853–1862.
- [45] Yudong Zhang, Xu Wang, Xuan Yu, Kuo Yang, Zhengyang Zhou, and Yang Wang. 2025. Do We Really Need GCNs in Traffic Forecasting? A Graph-Less Pure-MLP Architecture. In *WWW*. 1529–1533.
- [46] Wentian Zhao, Yanyun Gao, Tingxiang Ji, Xili Wan, Feng Ye, and Guangwei Bai. 2019. Deep Temporal Convolutional Networks for Short-Term Traffic Flow Forecasting. *IEEE Access* 7 (2019), 114496–114507.
- [47] Yusheng Zhao, Xiao Luo, Wei Ju, Chong Chen, Xian-Sheng Hua, and Ming Zhang. 2023. Dynamic Hypergraph Structure Learning for Traffic Flow Forecasting. In *ICDE*. 2303–2316.
- [48] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. 2020. GMAN: A Graph Multi-Attention Network for Traffic Prediction. In *AAAI*. 1234–1241.
- [49] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting. In *ICML*, Vol. 162. 27268–27286.

Appendix

A Notation Table

The notations used in this paper is provided in Table 4.

Table 4: Summary of key notations used in this paper.

Notation	Description
<i>General Notations and Problem Definition</i>	
$\mathcal{G}, \mathcal{V}, \mathcal{E}$	Road network graph, node set, and edge set.
N	The total number of nodes (sensors).
T_{in}, T_{out}	Length of the input sequence and output prediction sequence.
B	Batch size.
$\mathbf{Y}, \hat{\mathbf{Y}}$	Ground truth and model prediction.
<i>Feature and Embedding Module</i>	
$\mathbf{X}^{\text{raw}}$	The raw traffic flow feature tensor.
$\mathbf{X}^{\text{time}}$	The temporal feature matrix.
$\mathbf{X}^{\text{period}}$	The static periodicity feature matrix.
$\mathbf{Y}_{\text{FFT}}$	The complex-valued frequency spectrum from FFT.
$\mathbf{s}^{\text{daily}}, \mathbf{s}^{\text{weekly}}$	Vectors of daily and weekly periodicity strengths.
$\mathbf{X}^{\text{fused}}$	The fused feature tensor after concatenation.
$\mathbf{X}$	The final output of the embedding module.
<i>Hypergraph Construction Module</i>	
$\mathcal{H}, \mathcal{E}_h$	A hypergraph and its set of hyperedges.
$\mathbf{H}$	The final (fused) hypergraph incidence matrix.
M	The total number of unique hyperedges.
$\mathbf{D}$	The physical distance matrix between nodes.
$\mathbf{S}$	The Mutual Information (MI) similarity matrix.
k	The number of hypergraph construction neighbors.
<i>Hypergraph Learning Module</i>	
l	The layer index in the HGC network.
$\mathbf{X}^{(l)}$	Node feature representation at layer l .
$\mathbf{D}_e$	The diagonal matrix of hyperedge degrees.
$\mathbf{Z}^{\text{gru}}, \mathbf{Z}^{\text{mlp}}$	Output from the GRU and temporal MLP.
$\mathbf{Z}^{\text{final}}$	The fused representation before final prediction.
<i>Optimization</i>	
$\mathcal{L}_{\text{pred}}$	The prediction loss.
$\mathcal{L}_{\text{consist}}$	The structural consistency loss.
$\mathbf{w}_{\text{hyper}}$	The learnable hyperedge weight vector.
$\mathbf{w}_{\text{prior}}$	The static prior weight vector (initialized to all ones).
μ	Hyperparameter balancing the two loss components.

B Baselines

The details about the baselines are listed as follows:

- (1) Traditional statistic-based methods:
 - HA [15]: This model predicts future values by calculating the average of historical data from the same period.
 - ARIMA [27]: As a fundamental time series analysis method, it is widely used for building prediction models.
- (2) Neural network methods without the spatial graph:
 - FC-LSTM [30]: This model is a classic recurrent neural network that utilizes LSTM [16] units with fully connected layers to capture dependencies in sequential data.
 - TCN [46]: This model effectively learns long-term dependencies in time series by stacking causal convolutional layers with exponential expansion factors.
 - GRU-ED [7]: A model built upon an encoder-decoder framework with Gated Recurrent Units (GRU), designed for multi-step forecasting tasks.
 - DSANet [17]: An effective method for multivariate time series forecasting, which integrates convolutional neural networks with a dual self-attention mechanism.
 - STID [29]: This model leverages spatio-temporal contextual information via a unique decoupled position encoding strategy, operating without graph convolutions.
- (3) Neural network methods using the spatial graph:
 - STGCN [41]: This model integrates graph convolution with 1D temporal convolutions to concurrently model spatial structures and temporal dynamics.
 - DCRNN [26]: It models spatial dependencies by incorporating diffusion convolution operations within its Gated Recurrent Unit (GRU) cells.
 - GWN [38]: This method introduces an adaptive adjacency matrix and fuses it with dilated convolutions and diffusion graph convolution for spatio-temporal modeling.
 - ASTGCN [14]: This model employs both spatial and temporal attention mechanisms to dynamically capture evolving spatio-temporal correlations.
 - STG2Seq [2]: Built on the sequence-to-sequence (Seq2Seq) architecture, it incorporates multiple gated graph convolution modules and an attention mechanism to achieve multi-step prediction.
 - Z-GCNETs [6]: This model introduces the concept of "zigzag persistence" to identify and track significant topological features that evolve over time in the data.
 - STGHNC [5]: This model uses a spatial-temporal gated hypergraph convolution network to capture both pairwise and high-order spatial interactions, and further employs a channel attention mechanism to model cross-type feature correlations.
 - GLMST [45]: This model challenges the necessity of GCNs by proposing a graph-less architecture composed of pure Multi-Layer Perceptron (MLP) blocks to capture spatial and temporal dependencies.
 - LightST [43]: This model propose a new knowledge distillation paradigm termed LightST that transfers spatial and temporal knowledge from a high-capacity teacher to a light-weight student.

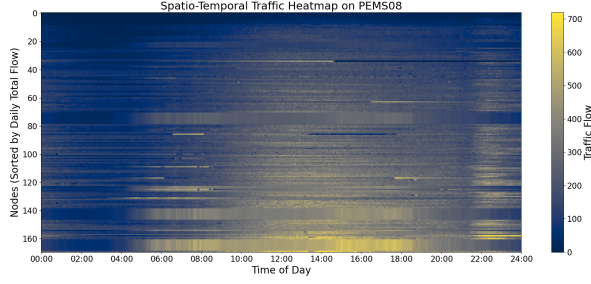


Figure 7: Spatio-temporal heterogeneity analysis.

C Dataset Analysis

In this section, we provide a visual analysis of the datasets to illustrate their key characteristics, which verifies the validity of our model.

C.1 Temporal Periodicity Analysis

As shown in Figure 6, we visualize the average traffic flow across all nodes for one week on both the PEMS08 and PEMS04 datasets. Several key patterns are evident. Firstly, both datasets exhibit strong **daily periodicity**, characterized by distinct morning and evening rush-hour peaks on weekdays. Secondly, a clear **weekly periodicity** is also observable, where the traffic patterns on weekends differ significantly from those on weekdays. These prominent and regular cyclical patterns underscore the importance of temporal context. This observation provides a strong justification for our *Periodicity-aware embedding module*, as it demonstrates the necessity of explicitly encoding instantaneous temporal features (day of week, time of day) and long-term periodic patterns (via FFT).

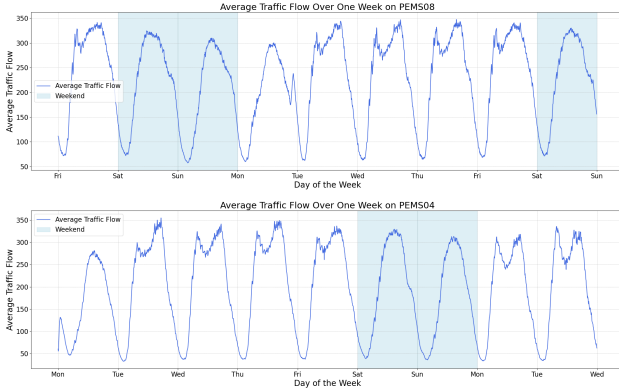


Figure 6: Temporal periodicity analysis.

C.2 Spatio-Temporal Heterogeneity Analysis

Figure 7 presents a heatmap of traffic flow for a typical day on the PEMS08 dataset. The y-axis represents the sensor nodes, sorted by their total daily traffic flow, while the x-axis represents the 288 time steps within the day. The plot reveals significant **spatial heterogeneity**. Horizontally, different nodes (rows) exhibit vastly different

traffic patterns. For example, nodes at the bottom of the plot are major arteries with consistently high traffic, while those at the top are minor roads with low traffic. This heterogeneity confirms that a one-size-fits-all approach is insufficient and justifies the need for node-specific modeling. Furthermore, the presence of similar horizontal patterns among different rows suggests the existence of functional correlations between nodes that may not be physically adjacent. This insight is the motivation for our *temporal hypergraph*, which is designed to capture these non-local relationships using Mutual Information.

C.3 Network Topology Analysis

The physical connectivity of the sensor networks for PEMS08 and PEMS04 is visualized in Figure 8. It is immediately apparent that the traffic sensors form a complex and **irregular graph structure**, rather than a simple grid. The nodes exhibit varying degrees of connectivity, with some acting as hubs and others being more peripheral. The distinct topologies of the two datasets also highlight the need for a model that can adapt to different graph structures. The existence of this underlying physical graph provides the fundamental rationale for employing a graph-based model. Specifically, it motivates the design of our *spatial hypergraph*, which explicitly leverages this topological information to model local spatial dependencies.

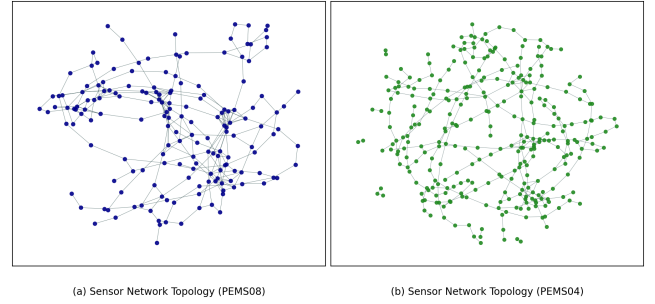


Figure 8: Network topology analysis.

D Hyper-parameter Analysis

We analyze the sensitivity of the hyperparameter μ , which balances the prediction loss and the structural consistency loss. We evaluate its value over the range $\{0.001, 0.01, 0.1, 1.0\}$ on the PEMS03 and PEMS08 datasets, with results shown in Figure 9. The experimental results show a consistent U-shaped trend for the error metrics: the model achieves its optimal performance across MAE, RMSE, and MAPE when μ is set to 0.01. As the value of μ increases beyond this point, a clear degradation in performance is observed. This is because a large μ value imposes an overly strong regularization, preventing the model from effectively learning the nuanced importance of different hyperedges by forcing the adaptive weights to adhere too strictly to their uniform prior. In summary, these results demonstrate that a moderate value for μ is crucial for balancing the learning of the adaptive structure with the primary goal of prediction accuracy.

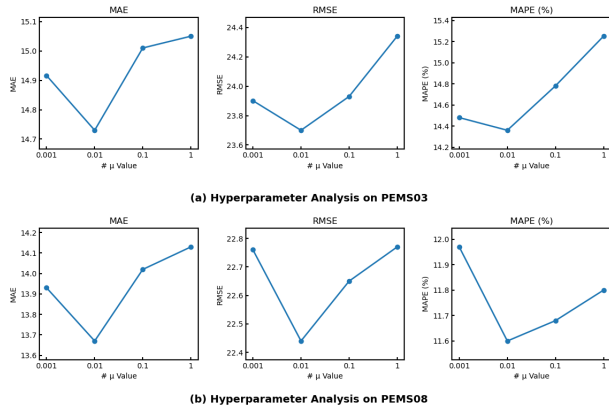


Figure 9: Hyperparameter sensitivity analysis of μ .

Figure 10 presents the hyperparameter analysis on the PEMS08 and PEMS04 datasets. We investigate the impact of the number of neighbors k , evaluating its value over the range $\{4, 8, 16, 32\}$. Based on the experimental results, we observe a clear trend: when the k is too small (for example, $k=4$), the performance is relatively poor. This is because the insufficient number of neighbors limits the model's ability to capture the rich spatio-temporal dependencies between nodes. As k increases to 8, the performance reaches or

approaches the optimal state on both datasets. However, when the k continued to increase to 32, the performance instead showed a significant downward trend. Because large k will introduce unrelated distant nodes as neighbors in the hypergraph, thereby causing noise interference and weakening the effective expression of local core associations instead. In summary, choosing a moderate k is key to achieving the best performance.

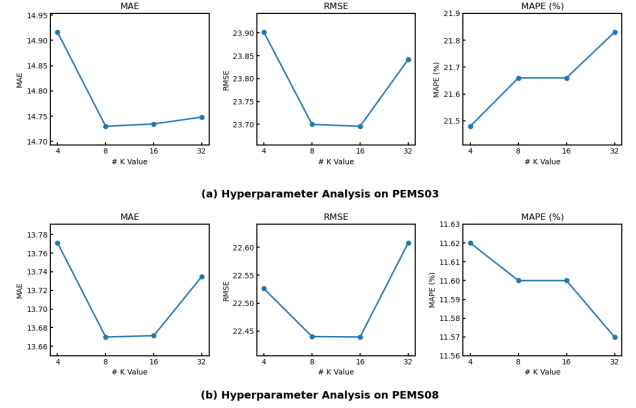


Figure 10: Hyperparameter sensitivity analysis of k .